

My Life in E-ink

Rohit Goswami · 2021-02-20 · workflow · tools · ramblings · books

Collection of odds and ends relating to e-readers including personal reminisces

Background

Reading has been a huge part of my life. The written word has had arguably more of an impact on my life than anything I have experienced in person. As a kid back in early 2000's; this meant a lot of library trips and saving for paperbacks. I also caught the first wave of the e-ink revolution. Nothing beats a real book, in terms of textures and scents; but e-ink devices and the fantastic tools outlined here should make reading digital books much more palpable [^fn:1].

I have been reading on my [Kobo Aura HD](#) for almost a decade now, ever since its release. This means my setup is about as stable as its going to get in the near future. As good a time as any to collect my thoughts [^fn:2]. The focus is on e-ink devices and auxiliary tools; not on all digital content; so there are no mentions of syncing or reading on the go (with a phone) or of monitors which are good for reading on.

The Content

In general; my e-ink reading habits can be broadly broken into the following categories:

Light Reading

Practically this includes [anything I review on Goodreads](#); these are not often re-read; nor read very deeply; since they are read for pleasure. They are however, rarely deleted

Required Reading

Anything which typically requires me to take notes or practice / write out proofs; these are most often considered to be either coursework (for someone somewhere) or research monographs. These are typically large (in size) and unwieldy (in that they often lack TOCs) and are read multiple times; with a focus on highlights and notes

Active Research

These are the most ephemeral of my reading habits; and also the most numerous; I do not typically store these on my e-reader; and rarely need to make notes on the reader [^fn:3]. These are often tiny; but require special work due to the metadata involved

Content Type	Software Stack	Deletion Rate
Light Reading	Calibre	Rare
Required Reading	Calibre	Never
Active Research	Calibre + Zotero	Frequent

Though I am a huge proponent of RSS feeds (with [Newsflash](#)) and read online content voraciously with both a [Pocket](#) and [Diigo](#) subscription[^fn:4]; I sincerely do not believe blog stuff or anything tailored for the web should have a presence on an e-ink device; so there shall be no mention of those parts of my reading habits[^fn:5].

Hardware

My primary e-reader is still my [Kobo Aura HD](#) (complete with a snazzy [hemp sleep-cover](#)), and has been my go to for almost a decade now since its release. Recently I have augmented my workflow with the [reMarkable 2](#); though I have yet to break it in very well; mostly because I tend to gravitate towards typing out my thoughts [^fn:6] instead of writing.

The Kobo Aura HD is still the pinnacle of reading technology to me; mostly because the firmware is easy to bypass; and there is an active community of developers on the [MobileRead Forums](#). Display and spec aside; the biggest reason for never replacing it has been the simple fact that most modern e-readers no longer support SD cards; and much of my workflow depends on storing insane amounts of material offline [^fn:7].

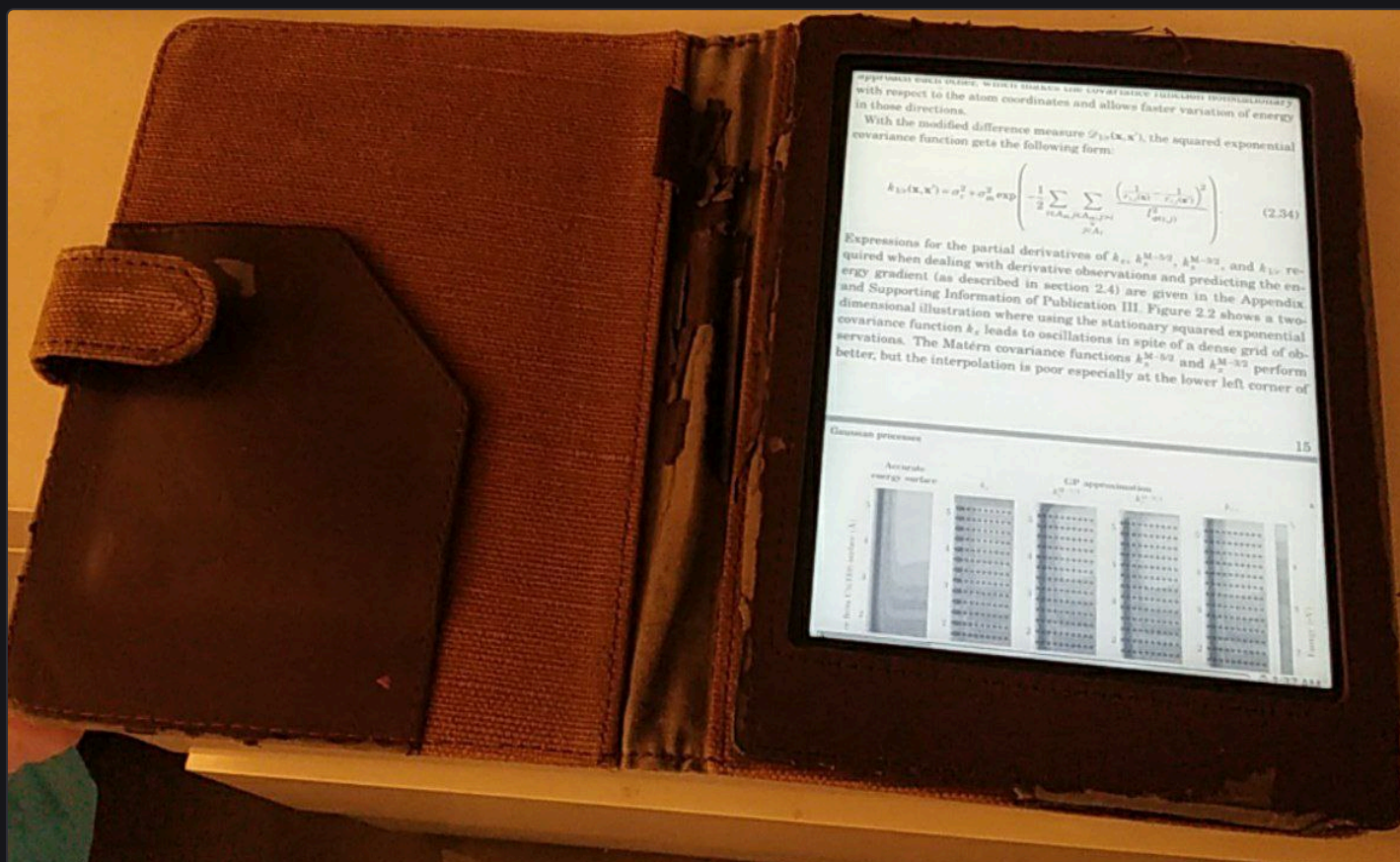


Figure 1: Primary reading device with Koreader

Personally, I never use Nickel (the default Kobo interface), and it would probably choke trying to scan my 200 GB of content; so I haven't updated the firmware in forever. My interactions are almost always in [Koreader](#); and my launching poison of choice is the now no longer developed [Kobo start menu](#) [^fn:8].

Software

Broadly speaking; the main parts of the software pipeline from digital book to brain are simply the syncing mechanism and the UI/UX/OS of the device in question; though it is often best to consider pre-processing books for devices too. These are covered in the order used.

k2pdfopt

The thought of reflowing text for an optimal reading experience, especially given the slightly limited processing power of my primary reading device is an enticing prospect. [K2pdfopt](#) or the [Kindle 2 PDF Optimizer](#) is as criminally underrated as it is fantastic. An approach which works well for my device involves setting up a simple shell script (part of my [Dotfiles](#)) for optimizing files on the fly before sending them through [calibre](#).

```
#!/usr/bin/env bash
# Get a filename
case "$#" in
0)
    echo "No arguments, so enter the filename, WITH the extension"
    read -p 'Document: ' docfile
    ;;
1)
    echo "OK, using the filename"
    docfile="$1"
    ;;
*)
    echo "Illegal number of parameters"
    exit
    ;;
esac

# Get basename
basename="${docfile%.*}"
ext="${docfile##*\.}"
echo "Basename ${basename} with $ext from $docfile"
echo "Making a local store for the outputs"
mkdir -p "$HOME/auraHDopt"

case "$ext" in
"djvu")
    echo "Converting djvu to pdf via ps and running k2pdfopt"
    djvups "${basename}.djvu" "${basename}.ps"
    ps2pdf "${basename}.ps" "${basename}.pdf"
    # The newline is for simulating the Enter key
    echo | k2pdfopt "${basename}.pdf" -wrap -hy -ws -0.2 -dev kbhd -x
    echo "Cleaning up"
    mv "${basename}_k2opt.pdf" "$HOME/auraHDopt"
    rm -rf "${basename}.{ps,pdf}"
    ;;
"pdf")
    echo "Converting pdf with gs and running k2pdfopt"
    gs -sDEVICE=pdfwrite -dCompatibilityLevel=1.4 -dPDFSETTINGS=/screen \
        -dNOPAUSE -dQUIET -dBATCH -sOutputFile="${basename}gs.pdf" "${basename}.pdf"
    echo | k2pdfopt "${basename}gs.pdf" -wrap -hy -ws -0.2 -dev kbhd -x
    echo "Cleaning up"
    rm "${basename}gs.pdf" -rf
    mv "${basename}gs_k2opt.pdf" "$HOME/auraHDopt"
    ;;
*)
    echo "Illegal file type"
    exit
    ;;
esac
```

The outputs can also be further processed with an [OCR \(Optical Character Recognition\)](#) script if required, and then edited in [Master PDF Editor](#) or something similar to add the table of contents interactively as well.

```
#!/bin/bash
# Use as find . -type f -name "*.pdf" -exec isOcr '{}' \;

# Shamelessly kanged from here:
# https://stackoverflow.com/questions/7997399/bash-script-to-check-pdfs-are-ocrd
# Only searches for text on the first 5 pages
# Modified to have red text. Also to possibly ocr the thing.

# -*- mode: shell-script-mode -*-

MYFONTS=$(pdffonts -l 15 "$1" | tail -n +3 | cut -d' ' -f1 | sort | uniq)
if [ "$MYFONTS" = '' ] || [ "$MYFONTS" = '[none]' ]; then
    echo "$(tput setaf 1)NOT OCR'ed: $1"
    if [[ -x "$(which ocrmypdf)" ]]; then
        echo "$(tput setaf 4)"
        echo "Converting to ${1%.*}_ocr.pdf with ocrmypdf"
        echo "$(tput setaf 7)"
        ocrmypdf --deskew --clean --rotate-pages \
            --jobs 4 -v --output-type pdfa "$1" "${1%.*}_ocr.pdf"
    elif [[ -x "$(which pypdfocr)" ]]; then
        echo "$(tput setaf 2) Looking for config files at $XDG_CONFIG_HOME/pypdfocr/config.yml"
        echo "$(tput setaf 3)"
        if [[ -e $XDG_CONFIG_HOME/pypdfocr/config.yml ]]; then
            echo "Using configuration settings"
            echo "$(tput setaf 4)"
            pypdfocr -c $XDG_CONFIG_HOME/pypdfocr/config.yml "$1"
        else
            echo "Using default settings"
            echo "$(tput setaf 4)"
            pypdfocr "$1"
        fi
        echo "$(tput setaf 2) You might want to get pypdfocr"
    fi
else
    echo "$1 is OCR'ed."
fi
```

The end result is:

- A directory with perfectly pdf files re-flowed text
 - Possibly OCR'ed for string searches

TOC editing is still rather janky; but this is also because the OCR process is still rather spotty.

Calibre

Calibre is an excellent library software, and there are very few alternatives which offer all the salient features:

Syncing

Apart from working well with a plethora of official devices, Koreader is also pretty well supported, and mounting folders allows for easy management of a secret library (e.g. .Library) on an SD card to prevent Nickel from reading and choking on large libraries

Multiple Libraries

I personally keep one for fiction, one for non-fiction, and one (transiently populated) one for papers

Good metadata collection

Nothing beats rich metadata, and with third party plugins, all the best content providers can be leveraged for blurbs; plus most purchased books come with metadata which calibre can read

It isn't perfect, there are far better OPDS (Open Publication Distribution System) servers like the fantastic COPS (Calibre OPDS) project, and there have been some security concerns in the past, but it is really usable and is under active development; plus it has a fun developer. I also personally find the file conversion lacking, compared to `k2pdfopt`, but as a library management system it is really good.

Zotero Sync

Calibre provides a handy [ZMI \(Zotero Metadata Importer\) plugin](#) which allows for exported papers to be imported into `calibre` and from then into the e-reader as expected. Combined with the folder mounts facilitated by `calibre` this allows for a painless way to ensure a quick export; optimize; sync; read and delete workflow. For those looking to [migrate from Mendeley](#) or [set up Zotero with WebDAV sync](#), [TurtleTech](#) has a good writeup on the process.

Koreader

Koreader is probably the best thing to happen to e-ink devices since sliced bread. It replaces the need to use any cables with an e-reader; since newer versions have a nice SSH server, and can also update itself. Since this is mostly used as is; and all the information required is on the [Github Wiki](#), there's not much else to say here.

It is probably worth noting that the in-built re-flow options do tend to cause major artifacts on older hardware, and is best avoided. Almost equivalently, and at a far lower cost in terms of performance, page contents can be fit to width and zoomed in automatically, which is almost as good as working with `k2pdfopt` in some special cases.

Conclusions

Given my unfortunate separation from my library back home; it is likely that my e-ink devices will continue to be my primary source of reading material. Plus the long retarded color e-ink market finally seems to be moving out of its stupor [^fn:9]. The only possible addendum to this methodology would probably involve integrating `orgmode` and the reMarkable 2 sometime. E-ink is here to stay. This setup would probably need revisions involving `rc1one` or `syncthing` if I ever gave up and opted for a "modern" device without an SD slot.